HierAberConic-D2C: A Physics-Grounded Hierarchical Memory Architecture for Non-Markovian Inference, Control, and Continual Learning

D. K. Ahorlu
Advanced Adaptive Systems Laboratory
correspondence@example.org

May 2026 | Technical Reference v1.0 | github.com/pilloverx/aberconics-framework

Abstract

We introduce **HierAberConic-D2C**, a hierarchical memory architecture for problems where the present state alone is insufficient to explain future evolution. The architecture is founded on three convergent principles: *(i)* Sum-of-Exponentials (SOE) memory kernels that make non-Markovian history explicit, interpretable, and provably stable; *(ii)* a predictive-coding hierarchy organised by timescale rather than abstraction alone, connecting levels via bottom-up error signals and top-down kernel modulation; and *(iii)* multi-timescale value learning in which each SOE memory channel induces its own discount horizon, unifying credit assignment and memory retention in a single dynamical substrate.

The framework is grounded in the Generalised Langevin Equation and the Mori–Zwanzig projection formalism, ensuring that memory kernels are not heuristic constructs but exact reductions of hidden degrees of freedom. A renormalisation flow connects levels of the hierarchy, preserving complete monotonicity and the SOE structure under coarse-graining. Spectral units—including effective memory dimension D_{eff}, memory capacity M_{cap}, and spectral entropy H_{mem}—serve as a universal diagnostic language, enabling real-time monitoring, pruning, and growth of memory channels.

The architecture targets non-Markovian settings: long-memory prediction, delayed control, continual adaptation, and hybrid symbolic–dynamical reasoning. A discrete-to-continuous bridge allows token or symbolic inputs to drive the continuous dynamical core without replacing it. Stability is guaranteed by design via proven conditions on kernel weights and decay rates. Early validation on coloured-noise approximation and Lorenz chaos suppression establishes the empirical foundation; broader benchmarks are underway. We position HierAberConic-D2C not as a replacement for existing deep learning but as a principled alternative for regimes where memory structure, interpretability, and stability guarantees are primary requirements.

Contents

1. Introduction

1.1. The Memory Problem in Modern AI

Contemporary sequence models—Transformers [5], LSTMs [6], and structured state-space models such as S4 and Mamba [7, 8]—have demonstrated remarkable empirical success on language, vision, and control benchmarks. Yet each encodes temporal history implicitly: attention weights are recomputed from scratch at every step, gated recurrent states carry no physically interpretable timescale, and no architecture provides a formal guarantee that memory structure is stable or recoverable from data.

This matters in a growing class of problems. Climate emulation must integrate forcing signals across days and decades simultaneously. Neural control systems must remember an action's consequence minutes after it was taken. Biomedical monitors must maintain patient baselines across non-stationary sessions. In each case the system is fundamentally *non-Markovian*: the present observation is insufficient to predict the future without access to structured history.

The central claim of this work is that *memory should be treated as an explicit computational object*, governed by physics, not inferred implicitly from data depth. The architecture we present formalises this claim across four dimensions: mathematical grounding, hierarchical organisation, value-directed adaptation, and hardware-mappable implementation.

1.2. What HierAberConic-D2C Is

HierAberConic-D2C is a hierarchical predictive-memory architecture built from **Aberconics Memory Blocks** (AMBs). Each AMB couples a latent state $u(t) \in \mathbb{R}^d$ to a bank of L SOE memory channels $\chi_\ell(t)$, whose decay rates γ_ℓ encode physical timescales and whose weights w_ℓ are learned online. The augmented state $z = (u, \chi)$ is Markovian in the extended space, making the system amenable to Bellman-style value learning despite describing non-Markovian dynamics in the observation space.

Multiple AMBs are stacked into a timescale hierarchy: lower levels handle fast sensory dynamics; higher levels accumulate slow structural context. Levels interact bidirectionally—lower levels send prediction errors upward; higher levels modulate the memory kernel of lower levels top-down. A renormalisation map governs how kernels transform across levels, preserving complete monotonicity.

A discrete-to-continuous bridge converts token or symbolic inputs into smooth forcing signals, allowing language and symbolic reasoning to be layered on top of the continuous dynamical core rather than replacing it. Training combines predictive-coding loss, multi-timescale TD value learning, and stability regularisation, orchestrated by a specialised runtime rather than a standard gradient loop.

1.3. Key Contributions

1. **Unified dynamical substrate.** A single mathematical object—the SOE memory kernel—simultaneously governs memory retention, credit assignment, and multi-timescale discounting.

2. **Stability by design.** Proven stability conditions (Theorem 4.1) are enforced throughout training. The completely monotone (CM) constraint on kernel weights is preserved under all update rules.

3. **Hierarchical renormalisation.** A formal renormalisation flow connects levels, with D_{eff} growing sub-multiplicatively— grounding the hierarchy in physical coarse-graining rather than engineering convention.

4. **Interpretable diagnostics.** Spectral units (D_{eff}, M_{cap}, H_{mem}) are computable at every step, enabling principled pruning, growth, and runtime monitoring.

5. **Hardware pathway.** Each memory channel maps directly to an RC circuit; the full stack approaches thermodynamic memory limits, providing a route to neuromorphic implementation.

6. **Domain-specific training runtime.** An Aberconics Director orchestrates stateful episodes, layerwise update rates, stability checks, and online consolidation—none of which are supported by standard ML training loops.

1.4. Scope and Limitations

The results presented here are a *foundation*, not a finished system. Early experiments on coloured-noise approximation and Lorenz chaos suppression establish that the mathematical framework

is implementable and yields physically meaningful results. These should be understood as proofs of concept, demonstrating that the design principles are sound and that the direction is productive. Broader benchmarks—chaotic forecasting at scale, delayed-reward control, long-context language modelling—are underway and will be reported in subsequent work.

HierAberConic-D2C does not compete with large-scale Transformers on tasks where massive data and compute are available. Its niche is the regime where memory structure, stability guarantees, and interpretability are primary: small data, continuous-time processes, safety-critical control, and scientific modelling.

1.5. Paper Organisation

Part I (§2–3) reviews prior work and establishes the mathematical foundations. Part II (§4–7) describes each architectural component in detail. Part III (§8–9) covers learning rules and the training runtime. Part IV (§10–11) provides the interpretability framework and full stability proofs. Part V (§12) reports experiments. Appendices provide pseudocode reference, hardware mapping, and a hyperparameter guide.

2. Background and Related Work

2.1. Non-Markovian Dynamics and Memory Kernels

A dynamical system is *non-Markovian* when its future evolution depends on history beyond the current state. The canonical mathematical treatment is the *Generalised Langevin Equation* (GLE) [1, 2]:

$$\dot{u}(t) = -\int_0^t K(t-s)\,\mathcal{L}[u(s)]\,\mathrm{d}s + f(t), \quad (1)$$

where $K(t)$ is the memory kernel, $\mathcal{L}$ is a spatial operator, and $f(t)$ is a fluctuating force. The Mori–Zwanzig formalism [1, 2] derives $K(t)$ rigorously by projecting high-dimensional Markovian dynamics onto an observable subspace. The kernel thereby encodes *all* information about eliminated degrees of freedom that is observable from the projected subspace.

2.2. Sum-of-Exponentials Approximation

Direct numerical convolution in (1) requires $\mathcal{O}(N_t^2)$ operations. The Sum-of-Exponentials approximation [3]

$$K(t) \approx K_L(t) = \sum_{\ell=1}^{L} w_\ell \mathrm{e}^{-\gamma_\ell t}, \qquad w_\ell, \gamma_\ell > 0, \quad (2)$$

introduces auxiliary variables $\chi_\ell(t) = \int_0^t \mathrm{e}^{-\gamma_\ell(t-s)}u(s)\,\mathrm{d}s$, each satisfying a local ODE, reducing complexity to $\mathcal{O}(L \cdot N_t)$ with $L \ll N_t$. Exponential convergence of rational Padé approximants guarantees high accuracy with modest $L \sim 5$–10 [4]. The stability analysis and corrected coupling formulations needed to make this computationally safe were established in prior work [20, 21] that forms the immediate foundation of this paper.

2.3. Structured State-Space Models

S4 [7] and its successors (Mamba [8], H3, Hyena) parameterise the state-space as a discretised linear dynamical system with structured matrices, achieving near-linear sequence complexity. These models share the SOE intuition but differ fundamentally: their parameters are tuned for computational efficiency rather than physical interpretability; they carry no stability guarantee; they do not support hierarchical timescale organisation; and their hidden state has no spectral diagnostic equivalent to D_{eff}. HierAberConic-D2C may be seen as a physically grounded, hierarchical extension of the SSM programme.

2.4. Predictive Coding

Predictive coding [11, 12] organises perception as hierarchical prediction–error minimisation. Each level predicts the activity of the level below; residual errors propagate upward as learning signals. The framework is biologically plausible and supports local learning rules. However, existing predictive-coding architectures do not incorporate explicit SOE memory channels, provide no timescale hierarchy grounded in physical decay rates, and do not address non-Markovian control. HierAberConic-D2C embeds predictive coding within an SOE dynamical substrate, adding memory, stability, and value-directed adaptation.

2.5. Neural ODEs and Continuous-Time Models

Neural ODEs [9] parameterise dynamics as $\dot{h} = f_\theta(h, t)$ with a neural network right-hand side. This provides a continuous-time foundation but offers no explicit memory structure, no stability guarantee, and no hierarchical timescale organisation.

Augmented variants [10] expand the state space, but without the physical motivation of the SOE–GLE connection. The present work may be seen as a Neural ODE with a principled, interpretable, and provably stable memory augmentation.

2.6. Multi-Timescale Reinforcement Learning

Temporal credit assignment—attributing a reward at time t to actions many steps earlier—is a central challenge in RL. Standard eligibility traces [13] decay at a single manually tuned rate λ. Options frameworks [14] introduce macro-actions but do not provide continuous-time multi-timescale credit assignment. We show (Section 6) that SOE memory channels provide a structured generalisation of eligibility traces, with physically motivated, per-channel decay rates that emerge from the memory kernel rather than from hyperparameter search.

3. Mathematical Foundations

3.1. Complete Monotonicity and Its Physical Basis

Definition 3.1. Complete Monotonicity

A function $K : [0,\infty) \to \mathbb{R}$ is *completely monotone* (CM) if it is infinitely differentiable and $(-1)^n K^{(n)}(t) \geq 0$ for all $t \geq 0$ and $n \geq 0$. Equivalently (Bernstein's theorem), K is CM if and only if it is the Laplace transform of a positive Borel measure μ on $[0,\infty)$:

$$K(t) = \int_0^\infty \mathrm{e}^{-\gamma t}\, \mathrm{d}\mu(\gamma). \tag{3}$$

At the molecular level, $K(t)$ is CM because it equals the autocorrelation of a Gaussian stationary bath, guaranteed by the fluctuation–dissipation theorem [1]. At higher levels of organisation—cells, tissues, regulatory networks—the bath is no longer in equilibrium and the standard proof fails. The following result recovers CM from a weaker, biologically relevant condition.

Theorem 3.1. CM from Stability

Let $u(t)$ satisfy a stable linear system near a fixed point with impulse-response kernel $K(t)$. If the linearised system is causal and passive (non-negative energy dissipation), then $K(t)$ is CM.

Proof sketch. $K(t)$ is the impulse response of a stable, causal, passive system. Stability places all poles in the open left half-plane; passivity ensures the transfer function $\hat{K}(s)$ has non-negative real part for $\mathrm{Re}(s) \geq 0$. By the positive-real lemma, $\hat{K}(s)$ is the Laplace transform of a positive measure on $[0,\infty)$, which is precisely the Bernstein representation (3). □

The biological corollary is direct: pathological states (cancer, neurodegeneration, cardiac arrhythmia) are precisely those in which stability breaks down, hence CM fails. Complete monotonicity is the *mathematical signature of functional biological organisation*, not merely a computational convenience.

3.2. SOE Embedding and the Auxiliary-Variable System

Replacing the continuous measure in (3) by a finite atomic measure $\mu = \sum_\ell w_\ell \delta_{\gamma_\ell}$ gives the SOE approximation (2). Defining auxiliary variables

$$\chi_\ell(t) \;=\; \int_0^t \mathrm{e}^{-\gamma_\ell (t-s)} u(s)\, \mathrm{d}s, \tag{4}$$

each χ_ℓ satisfies the local ODE

$$\dot{\chi}_\ell = -\gamma_\ell \chi_\ell + u(t), \qquad \ell = 1, \ldots, L, \tag{5}$$

and the non-local convolution becomes the finite sum

$$\int_0^t K(t-s) u(s)\, \mathrm{d}s \;\approx\; \sum_\ell w_\ell \chi_\ell(t). \tag{6}$$

The augmented state $\boldsymbol{z} = (u, \chi_1, \ldots, \chi_L)$ evolves as a *Markovian* ODE system in $\mathbb{R}^{1+L}$, despite describing non-Markovian dynamics in u alone. This Markovianisation of memory is the computational engine of the entire framework.

3.3. Nonlinearity–Memory Duality

The following result, established in the geometric-Fourier-extension line of work [21], is the theoretical keystone of the framework.

Theorem 3.2. Nonlinearity–Memory Duality

(i) *Reduction.* Any Lipschitz nonlinear Markovian system on a high-dimensional space, projected onto a lower-dimensional observable

subspace, generates a linear non-Markovian system with a CM memory kernel. The kernel encodes all information about eliminated degrees of freedom that is observable from the subspace.

(ii) Embedding. Any linear non-Markovian system with CM kernel $K(t)$ embeds exactly into a higher-dimensional Markovian system via the SOE auxiliary construction. The embedding is exact when L is finite and $K(t)$ has a finite atomic spectral measure.

Corollary 3.1. Completeness of Non-Markovian Description

The non-Markovian kernel description is not an approximation of the Markovian one: it is exactly equivalent, with $K(t)$ carrying all discarded information. Fitting $K(t)$ from data is an act of *model identification*, not phenomenological curve-fitting.

3.4. Renormalisation Flow Across Levels

Moving from a fine-grained level n to a coarser level $n+1$ involves integrating out fast degrees of freedom. The resulting kernel satisfies the renormalisation map:

$$K_{n+1}(t) = (K_n * K^n_{\text{bath}})(t) + K^n_{\text{direct}}(t), \tag{7}$$

where $*$ denotes temporal convolution, K^n_{bath} is the bath correlation function at level n, and K^n_{direct} captures direct memory contributions that survive coarse-graining.

Proposition 3.1. CM Preserved Under Coarse-Graining

If $K_n(t)$ and $K^n_{\text{bath}}(t)$ are both CM, then $K_n * K^n_{\text{bath}}$ is CM.

Proof. The Bernstein class is closed under convolution because convolution of Laplace transforms is multiplication: the product of two completely monotone Laplace transforms is completely monotone. □ □

When both kernels are SOE, their convolution is also SOE:

$$\left(K_n * K^n_{\text{bath}}\right)(t) = \sum_{\ell}\sum_{m} \frac{w^n_\ell u^n_m}{\gamma^n_\ell - \delta^n_m}\left(e^{-\delta^n_m t} - e^{-\gamma^n_\ell t}\right), \tag{8}$$

yielding at most $L \times M$ terms, reduced by pruning to $\tilde{L}$ significant modes. The effective dimension transforms as

$$D^{n+1}_{\text{eff}} \;\le\; D^n_{\text{eff}} \cdot D_{\text{eff}}(K^n_{\text{bath}}), \tag{9}$$

establishing sub-multiplicative growth of memory complexity across levels, consistent with observed moderate D_{eff} values at each biological scale.

4. The Aberconics Memory Block

4.1. Core Equations

An **Aberconics Memory Block** (AMB) at level n couples a latent state $u_n(t) \in \mathbb{R}^d$ to L_n memory channels $\chi_{n,\ell}(t)$ via:

$$\dot{u}_n = -\ell_n u_n - \sum_{\ell=1}^{L_n} w_{n,\ell}\,\chi_{n,\ell} + \mathcal{F}_n(u_{n-1}) + \mathcal{P}_n(u_{n+1}) \cdot u_n + x_n \tag{10}$$

$$\dot{\chi}_{n,\ell} = -\gamma_{n,\ell}\,\chi_{n,\ell} + u_n, \tag{11}$$

where $\ell_n > 0$ is the leak rate, $\mathcal{F}_n$ is the bottom-up forcing from level $n-1$, $\mathcal{P}_n$ is top-down kernel modulation from level $n+1$, and $x_n(t)$ is direct external input (non-zero only at the base level). All learnable parameters—$w_{n,\ell}$, $\gamma_{n,\ell}$, ℓ_n—are strictly positive, preserving CM.

4.2. Three Stable Coupling Formulations

Early formulations used state-driven positive feedback, which is provably unstable for typical parameters [20]. Three corrected formulations are used:

Formulation A (Input-Driven).

$$\dot{u} = -\ell u + \textstyle\sum_\ell w_\ell \chi_\ell + x(t), \qquad \dot{\chi}_\ell = -\gamma_\ell \chi_\ell + x(t). \tag{12}$$

Memory channels track external input, not the state. The Jacobian is upper-triangular, giving eigenvalues $\{-\ell, -\gamma_1, \ldots, -\gamma_L\}$—all negative, hence unconditionally stable.

Formulation B (Negative Feedback).

$$\dot{u} = -\ell u - \textstyle\sum_\ell w_\ell \chi_\ell + x(t), \qquad \dot{\chi}_\ell = -\gamma_\ell \chi_\ell + u. \tag{13}$$

Memory provides intelligent dissipation. Used for hierarchy levels and chaos suppression.

Formulation C (Second-Order Resonant).

$$\ddot{u} + 2\zeta\omega_0 \dot{u} + \omega_0^2 u = x(t) + \textstyle\sum_\ell w_\ell \chi_\ell, \quad \dot{\chi}_\ell = -\gamma_\ell \chi_\ell + u. \tag{14}$$

A damped oscillator with memory-modulated forcing, enabling resonant echoes when $1/\gamma_\ell \approx 1/\omega_0$.

4.3. Stability Theorem

Theorem 4.1. Positive-Feedback Instability (Formulation B corrected).

Consider Formulation B with $x(t) = 0$. The equilibrium $(z) = \mathbf{0}$ is asymptotically stable if and only if

$$\ell > \sum_{\ell=1}^{L} \frac{w_\ell}{\gamma_\ell}. \tag{15}$$

Proof. The Jacobian of the linearised system is

$$J = \begin{pmatrix} -\ell & -w_1 & \cdots & -w_L \\ 1 & -\gamma_1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 1 & 0 & \cdots & -\gamma_L \end{pmatrix}.$$

At $\lambda = 0$, the characteristic polynomial evaluates to $\det(-J)\big|_{\lambda=0} = \ell \prod_\ell \gamma_\ell (1 - \sum_\ell w_\ell/(\ell\gamma_\ell))$. This is positive (no zero eigenvalue) iff condition (15) holds. A Lyapunov function $V = \frac{1}{2}u^2 + \frac{1}{2}\sum_\ell (w_\ell/\gamma_\ell)\chi_\ell^2$ satisfies $\dot{V} \le 0$ when (15) holds, confirming asymptotic stability. □ □

This condition is enforced at every training step, providing a *hard stability guarantee* absent from all standard recurrent architectures.

4.4. Hardware Mapping

Each memory channel χ_ℓ with decay rate γ_ℓ maps to an RC circuit with time constant $\tau_\ell = 1/\gamma_\ell$: $R = 100\,\mathrm{k\Omega}$, $C = 1/(\gamma_\ell \cdot R)$. A summing operational amplifier realises the weighted output $\sum_\ell w_\ell \chi_\ell$. The Landauer bound for maintaining $L = 7$ channels at $T = 300\,\mathrm{K}$ and $f = 1\,\mathrm{kHz}$ is $P_{\min} \approx 1.4 \times 10^{-15}\,\mathrm{W}$—approaching the thermodynamic limit and three orders of magnitude below SRAM [20, 18].

5. The Hierarchical MIN Architecture

5.1. Design Principle: Timescale as Hierarchy

Standard deep networks organise layers by abstraction. HierAberConic-D2C organises levels by *timescale*: level n has characteristic timescale $\bar{\tau}_n = 1/\bar{\gamma}_n$ (geometric mean of its channel decay rates), with the constraint

$$\bar{\tau}_1 < \bar{\tau}_2 < \cdots < \bar{\tau}_N. \tag{16}$$

Lower levels capture fast sensory dynamics; higher levels accumulate slow structural context. This organisation is not a modelling assumption: it emerges from the renormalisation flow (7), which shifts spectral weight toward slower modes at each coarser level.

A reference parameterisation for language or continuous-time control is given in Table 1.

Table 1. Reference level parameterisation.

Level	Role	$\bar{\tau}_n$	L_n	D_{eff} target
1	Fast sensory	1–10 ms	3–5	2–4
2	Feature	50–200 ms	5–8	4–7
3	Context	0.5–2 s	8–12	6–10
4	Semantic	5–30 s	8–12	6–10
5	Structural	1–5 min	5–8	5–8
6	Document/Goal	10–30 min	3–5	3–5

5.2. Bottom-Up Coupling

Level $n-1$ drives level n through a forcing function:

$$\mathcal{F}_n(u_{n-1}) = h_n\big(u_{n-1}(t)\big), \tag{17}$$

where $h_n : \mathbb{R}^d \to \mathbb{R}^d$ is a learned projection (linear for interpretability; nonlinear optionally). This corresponds to the standard Mori–Zwanzig coarse-graining and preserves CM at level n provided $K_{n-1}(t)$ is CM (Proposition 3.1).

5.3. Top-Down Kernel Modulation

The key architectural innovation is that higher levels reshape the *memory structure* of lower levels, not merely their activity. Level n modulates the kernel of level $n-1$ by making weights and decay rates functions of the slow state:

$$K_{n-1}(t;\, u_n) = \sum_\ell w_\ell(u_n)\, \mathrm{e}^{-\gamma_\ell(u_n)\, t}. \tag{18}$$

CM is preserved by parameterising

$$w_\ell(u_n) = \mathrm{e}^{\alpha_\ell(u_n)}, \qquad \gamma_\ell(u_n) = \mathrm{e}^{\beta_\ell(u_n)}, \tag{19}$$

with unconstrained α_ℓ, β_ℓ (linear projections of u_n). A global stability bound $w_\ell(u_n) \le w_{\max}$ ensures condition (15) holds for all u_n.

5.4. The Coupling Assembly Rule

The complete bidirectional coupling at each timestep is:

$$\text{Level } n \xrightarrow{\text{forcing}} \text{Level } n-1: \quad \mathcal{F}_{n-1}(t) \leftarrow g_n\big(u_n(t)\big), \tag{20}$$

$$\text{Level } n \xrightarrow{\text{modulation}} \text{Level } n-1: \quad K_{n-1}(t) \leftarrow K_{n-1}(t;\, u_n(t)), \tag{21}$$

$$\text{Level } n-1 \xrightarrow{\text{output}} \text{Level } n: \quad \mathcal{F}_n(t) = h_n\big(u_{n-1}(t)\big), \tag{22}$$

with CM preservation enforced: $w_\ell(u_n) \geq 0$, $\gamma_\ell(u_n) > 0$ for all u_n. Coupling functions g_n, h_n are learned from cross-level correlations via local Hebbian rules (Section 8).

5.5. Predictive Coding Integration

Each level n maintains a prediction of level $n-1$:

$$\hat{y}_n(t) = W_{\text{pred},n}\, u_n(t), \tag{23}$$

and receives the prediction error

$$\varepsilon_n(t) = u_{n-1}(t) - \hat{y}_n(t). \tag{24}$$

Errors propagate bottom-up; predictions propagate top-down. Learning at level n is driven by ε_n: weights $w_{n,\ell}$ are updated to reduce future prediction errors, while timescales $\gamma_{n,\ell}$ are adapted to match the temporal structure of the error signal (Section 8).

6. Multi-Timescale Value Learning

6.1. The Augmented State and Its Markov Property

A critical correction from standard RL practice: the value function V is *not* part of the dynamical state. The state is:

$$z_n(t) = \big(u_n(t),\, \boldsymbol{\chi}_n(t)\big) \in \mathbb{R}^{d+L_n}, \tag{25}$$

and the critic $V_\ell : \mathbb{R}^{d+L_n} \to \mathbb{R}$ is a learned function *over* z_n. Including V in the state would introduce a circular dependency; the two objects must remain cleanly separated.

Proposition 6.1. Markov Property of Augmented State

The process $(z_n(t))_{t\geq 0}$ is Markovian: given $z_n(t)$, the future evolution is determined by the dynamics (10)–(11) and requires no additional history.

Proof. $\chi_{n,\ell}(t) = \int_0^t \mathrm{e}^{-\gamma_\ell(t-s)} u_n(s)\, \mathrm{d}s$ is fully determined by $z_n(t)$ and the subsequent ODE, with no dependence on history before t. □ □

This result justifies Bellman-style value learning over z_n despite the non-Markovian nature of u_n alone.

6.2. Hamilton–Jacobi–Bellman Equation

The value function for the augmented state satisfies the continuous-time HJB equation:

$$\rho V(z) = \max_a \Big[R(z,a) + \nabla_u V \cdot f(u, \boldsymbol{\chi}, a) + \sum_\ell \frac{\partial V}{\partial \chi_\ell}\big(-\gamma_\ell \chi_\ell + u\big) \Big], \tag{26}$$

where $\rho > 0$ is a global discount rate and f encodes the latent dynamics (10). The memory channels appear explicitly in (26) through two gradient terms:

- *Decay term:* $-\gamma_\ell \chi_\ell\, \partial V/\partial \chi_\ell$ (fading memory reduces value sensitivity);
- *Update term:* $u\, \partial V/\partial \chi_\ell$ (current state refreshes memory relevance).

Slow channels (small γ_ℓ) contribute more to long-horizon value; fast channels dominate immediate assessment.

6.3. Per-Channel Value Decomposition

We define a *per-channel value function* with discount rate γ_ℓ matching the memory channel:

$$V_\ell(z) = \mathbb{E}\left[\int_t^\infty \mathrm{e}^{-\gamma_\ell(s-t)} r(z(s), a(s))\, \mathrm{d}s \,\middle|\, z(t) = z \right]. \tag{27}$$

The aggregate value under the full kernel is

$$V_K(z) = \sum_\ell w_\ell\, V_\ell(z), \tag{28}$$

which follows directly from the SOE decomposition of the reward functional. We adopt the ensemble of L critics as a design choice (rather than a single critic on z or a shared-feature critic), motivated by the natural correspondence between channel ℓ and horizon $1/\gamma_\ell$.

Remark 6.1. *The SOE decay rates γ_ℓ induce a natural family of timescale-matched discount factors $e^{-\gamma_\ell \Delta t}$. This is a structural correspondence, not a theorem: exact equivalence between memory discount and reward discount holds when the reward functional shares the same SOE kernel as the memory.*

6.4. Per-Channel TD Errors

Discretising (27) with timestep Δt:

$$\delta_\ell(t) = r(t) + e^{-\gamma_\ell \Delta t} V_\ell(z_{t+\Delta t}) - V_\ell(z_t). \quad (29)$$

Each δ_ℓ has variance $\mathrm{Var}[\delta_\ell] \propto 1/\gamma_\ell$: slow channels have noisier TD targets because they integrate over longer, more uncertain horizons. Per-channel learning rates are therefore set as

$$\eta_\ell^V = \eta_{\text{base}}^V \cdot \frac{\mathrm{SNR}_\ell}{\max_{\ell'} \mathrm{SNR}_{\ell'}}, \quad (30)$$

where $\mathrm{SNR}_\ell = \mathbb{E}[|\delta_\ell|]/\mathrm{Std}[\delta_\ell]$ is the signal-to-noise ratio of the TD error for channel ℓ. This is presented as a practical heuristic: the optimal schedule depends on reward sparsity and horizon length.

6.5. Eligibility Traces as a Structured Generalisation

The memory channels $\chi_\ell(t)$ provide a *structured generalisation of eligibility traces*. Standard TD(λ) traces $e_t = \lambda\gamma e_{t-1} + \nabla_w V(s_t)$ operate in weight space with a single, manually tuned decay rate λ. The channels χ_ℓ operate in observation space with L physically motivated decay rates, supporting simultaneous multi-timescale credit assignment without hyperparameter search.

The three-factor update rule (Section 8) exploits this directly: the value gradient at timescale ℓ is carried by χ_ℓ, which has already accumulated the relevant history at the matching timescale.

7. Discrete-to-Continuous Bridge

7.1. Design Principle

The continuous dynamical core is primary; discrete inputs are a boundary condition, not the substrate. Tokens or symbols are converted into smooth forcing signals that drive the base-level AMB. Between tokens, the system evolves freely under its own dynamics, integrating context in the memory channels. This allows the model to *think between words*—an operation impossible in architectures where processing is token-synchronous.

7.2. Token-to-Forcing Conversion

A token k arriving at time t_k is embedded as $\boldsymbol{e}_k = \mathrm{Embed}(k) \in \mathbb{R}^d$, then spread into a smooth forcing signal:

$$x(t) = \boldsymbol{e}_k \cdot \phi(t - t_k), \qquad \phi(\tau) = \sum_{j=1}^{J} \alpha_j e^{-\beta_j \tau}, \quad (31)$$

where α_j, β_j are learnable parameters defining the injection kernel. This is itself an SOE, ensuring the forcing is smooth, physically motivated, and integrates naturally with the memory channels.

7.3. Continuous-to-Discrete Readout

At discrete output times t_k, logits are read from the base-level state:

$$\ell_k = W_{\text{out}}\, u_1(t_k) + b_{\text{out}}, \quad (32)$$

passed through a softmax (or Gumbel–Softmax for differentiable discrete sampling [15]) to produce a token distribution. Diffusion or flow-matching modules may be attached as generative adapters for structured output spaces.

7.4. Language as a Boundary Mechanism

Language and symbolic processing are *boundary mechanisms* layered on the dynamical substrate, not the substrate itself. This distinction has practical consequences: the continuous memory accumulates cross-token context in $\chi_\ell(t)$ without a fixed context window; information about token k is available at token $k{+}m$ through the slow channels, with graceful exponential decay rather than hard truncation. Long-distance dependencies are handled by channels with small γ_ℓ; local syntax by channels with large γ_ℓ.

8. Learning Rules and Adaptation

8.1. The Three-Factor Update Rule

At each level n, kernel weights $w_{n,\ell}$ are updated by a three-factor rule combining predictive coding and value signals:

$$\frac{dw_{n,\ell}}{dt} = \eta_{\text{pred}}\, \chi_{n,\ell}\, \varepsilon_n + \eta_{\text{val}}\, \chi_{n,\ell}\, \delta_{n,\ell} - \beta(\varepsilon_n)\, w_{n,\ell}, \quad (33)$$

where:

- $\eta_{\text{pred}}\, \chi_{n,\ell}\, \varepsilon_n$ is the Hebbian prediction term: strengthen channels that correlate with prediction error;

- $\eta_{\text{val}}\,\chi_{n,\ell}\,\delta_{n,\ell}$ is the value term: strengthen channels that carry credit for reward;
- $\beta(\varepsilon_n) = \beta_0/(1+|\varepsilon_n|)$ is error-modulated decay—faster forgetting when predictions are accurate (consolidation), slower when errors are large (plasticity).

The CM constraint $w_{n,\ell} \geq 0$ is enforced after each step.

8.2. Timescale Adaptation

Decay rates $\gamma_{n,\ell}$ adapt at a rate $\alpha \ll \eta_{\text{pred}}, \eta_{\text{val}}$ (timescales change 10–100× slower than weights):

$$\frac{\mathrm{d}\gamma_{n,\ell}}{\mathrm{d}t} = -\alpha\,\gamma_{n,\ell}\,\mathrm{sign}\big(\chi_{n,\ell}\cdot\varepsilon_n\big), \tag{34}$$

with hard constraints $\gamma_{\min} \leq \gamma_{n,\ell} \leq \gamma_{\max}$ enforced after each update. Intuitively: if $\chi_{n,\ell}$ and ε_n are positively correlated, this channel is relevant to the error and should integrate over a longer window (decrease γ); if negatively correlated, it should release memory faster (increase γ).

8.3. Consolidation Mechanism

To prevent catastrophic forgetting, weights are split into fast and slow components:

$$w_\ell = w_\ell^{\text{fast}} + w_\ell^{\text{slow}}, \tag{35}$$

where w_ℓ^{fast} is updated by (33) at every step, and w_ℓ^{slow} is updated periodically:

$$w_\ell^{\text{slow}} \leftarrow w_\ell^{\text{slow}} + \beta_c\big(w_\ell^{\text{fast}} - w_\ell^{\text{slow}}\big), \tag{36}$$

with consolidation rate $\beta_c \ll \beta_0$. Fast weights then reset toward the slow baseline: $w_\ell^{\text{fast}} \leftarrow (1-\beta_c)w_\ell^{\text{fast}} + \beta_c\, w_\ell^{\text{slow}}$. This mechanism, analogous to sleep-dependent memory consolidation [17], prevents recent experience from overwriting long-term structure.

8.4. Channel Birth and Death

The number of channels L_n is adaptive:

Death (pruning). If $w_{n,\ell} < \epsilon_{\text{prune}} \cdot \max_{\ell'} w_{n,\ell'}$ for more than T_{prune} steps, channel ℓ is removed.

Birth (growth). If $D_{\text{eff},n} \approx L_n$ (all channels contribute equally, saturation) and the prediction error ε_n contains significant power at a timescale $\hat{\tau}$ not covered by current channels, a new channel is inserted with $\gamma_{\text{new}} = 1/\hat{\tau}$ and small initial weight. These operations preserve CM and the stability condition (15) by construction.

8.5. Online versus Offline Training

Online mode. Dynamics run continuously; weights updated by (33) at every step; value heads updated when sufficient trace data is available. No batching; no gradient accumulation. Suitable for deployment and real-time adaptation.

Offline mode. Trajectory windows are sampled from a TraceStore (Section 9); weight updates via (33) applied to stored traces; value head parameters updated by gradient descent on $\sum_\ell \delta_\ell^2$. Suitable for initial training and benchmarking.

Hybrid mode. Continuous online dynamics with periodic offline value-learning passes on stored traces. This is the recommended configuration: the dynamical substrate never stops; learning operates at two speeds.

9. The Aberconics Training Runtime

9.1. Why a Specialised Runtime Is Required

Standard ML training loops assume a static computation graph, i.i.d. mini-batches, a single global loss, and stateless processing between batches. HierAberConic-D2C violates all four assumptions: memory channels are stateful across episodes; training must support layerwise updates at different rates; the learning objective is a composite of local prediction errors, per-channel TD errors, stability penalties, and memory regularisation; and the system must support online consolidation. A domain-specific training runtime—an *Aberconics Director*—is therefore required.

The runtime is structured as three layers:

1. **C++ core** (GFE kernel engine, ABERSOE runtime, hierarchical MIN): handles dynamics, kernel fitting, spectral diagnostics, stability monitoring, and serialisation.
2. **Python director** (AberconicsDirector, TraceStore, Scheduler, SpectralInspector, CheckpointManager): handles orchestration, objective composition, training phase transitions, and logging.
3. **Optional accelerator**: custom CUDA kernels for batched SOE stepping, distributed execution (future work).

9.2. The AberconicsDirector State Machine

The Director operates as a finite-state machine governing training phases:

Algorithm 1 AberconicsDirector state machine

States: WARMUP, EXPLORE, CONSOLIDATE, EVALUATE, PRUNE, GROW
Initialise: state $\leftarrow$ WARMUP, $t \leftarrow 0$
while not done **do**
  $z, \varepsilon \leftarrow$ HIERARCHYSTEP(obs, Δt)
  Append (z, r, ε) to TraceStore
  if state = WARMUP and $t > T_{\text{warmup}}$ **then**
    state $\leftarrow$ EXPLORE
  else if state = EXPLORE **then**
    UPDATEWEIGHTS(ε, δ, Δt)
    if $\bar{\varepsilon} < \varepsilon_{\text{thresh}}$ **then**
      state $\leftarrow$ CONSOLIDATE
    end if
  else if state = CONSOLIDATE **then**
    CONSOLIDATE(β_c)
    if consolidation timer expired **then**
      state $\leftarrow$ EXPLORE
    end if
  end if
  if $D_{\text{eff}} < D_{\min}$ **then** state $\leftarrow$ PRUNE
  end if
  if $D_{\text{eff}} \approx L$ **then** state $\leftarrow$ GROW
  end if
  if $t \bmod T_{\text{eval}} = 0$ **then** EVALUATE
  end if
  STABILITYCHECK
  $t \leftarrow t + \Delta t$
end while

9.3. The TraceStore

The TraceStore is not a standard replay buffer. Random i.i.d. sampling destroys temporal correlations that are the reason the architecture exists. Instead:

Contiguous sampling ensures that $\chi(t)$ in the sampled window reflects the actual memory state at that time, not an incoherent mixture of unrelated episodes.

Algorithm 2 TraceStore: contiguous-window trajectory memory

Input: Capacity C, window size W, augmented-state dimension $d+L$
Allocate circular buffers for $(u, \chi, r, \varepsilon)$
procedure ADD(u, χ, r, ε)
  Write to position ptr; ptr $\leftarrow$ (ptr $+ 1$) mod C
end procedure
procedure SAMPLEWINDOW
  $s \leftarrow \text{Uniform}(0, \min(|\text{stored}|, C) - W)$
  return contiguous slice $[s, s + W)$ ▷ Preserves temporal correlation
end procedure

Algorithm 3 StabilityMonitor: Theorem 4.1 enforcement

procedure CHECK(level n)
  $\rho_n \leftarrow \sum_\ell w_{n,\ell} / (\gamma_{n,\ell} \cdot \ell_n)$
  if $\rho_n > 0.9$ **then**
    $w_{n,\ell} \leftarrow w_{n,\ell} \cdot (0.8/\rho_n)$ ▷ Rescale to satisfy (15)
    Log violation
    return VIOLATED
  end if
  return OK
end procedure
procedure EMERGENCYSTABILISE(all levels)
  for each level n **do**
    while CHECK(n) = VIOLATED **do**
      Reduce $w_{n,\ell}$ by 10%
    end while
  end for
end procedure

9.4. The StabilityMonitor

9.5. Composite Training Objective

The total training loss for an offline pass over a sampled window is:

$$\mathcal{L} = \lambda_{\text{task}}\mathcal{L}_{\text{task}} + \lambda_{\text{pred}}\mathcal{L}_{\text{pred}} + \lambda_{\text{val}}\mathcal{L}_{\text{val}} + \lambda_{\text{stab}}\mathcal{L}_{\text{stab}} + \lambda_{\text{mem}}\mathcal{L}_{\text{mem}}, \tag{37}$$

where

$$\mathcal{L}_{\text{pred}} = \sum_n \mathbb{E}_t[\varepsilon_n(t)^2], \tag{38}$$

$$\mathcal{L}_{\text{val}} = \sum_n \sum_\ell \mathbb{E}_t[\delta_{n,\ell}(t)^2], \tag{39}$$

$$\mathcal{L}_{\text{stab}} = \sum_n \text{ReLU}\Big(\sum_\ell \frac{w_{n,\ell}}{\gamma_{n,\ell}} - 0.9\,\ell_n\Big), \tag{40}$$

$$\mathcal{L}_{\text{mem}} = \sum_n \text{ReLU}(D_{\text{eff},n} - D_{\max,n}) - \lambda_{\text{div}}\,\text{Var}_\ell\big(\log \gamma_{n,\ell}\big). \tag{41}$$

The diversity penalty $-\lambda_{\text{div}}\,\text{Var}(\log \gamma_{n,\ell})$ prevents timescale collapse—a key failure mode where all channels converge to the same decay rate and lose the multi-timescale structure that makes the architecture distinct.

10. Spectral Diagnostics and Interpretability

10.1. Spectral Units: A Universal Diagnostic Language

Given a fitted SOE kernel with weights $\{w_\ell\}$ and rates $\{\gamma_\ell\}$, the normalised weights are $\tilde{w}_\ell = w_\ell / \sum_{\ell'} w_{\ell'}$. The spectral units are:

$$M_{\text{cap}} = \frac{\sum_\ell \tilde{w}_\ell/\gamma_\ell^2}{\sum_\ell \tilde{w}_\ell/\gamma_\ell} \quad \text{(mean memory depth, seconds)}, \tag{42}$$

$$M_{\text{scale}} = \log_{10}(\gamma_{\max}/\gamma_{\min}) \quad \text{(timescale span, decades)}, \tag{43}$$

$$M_{\text{res}} = L/M_{\text{scale}} \quad \text{(modes per decade)}, \tag{44}$$

$$H_{\text{mem}} = -\sum_\ell \tilde{w}_\ell \ln \tilde{w}_\ell \quad \text{(spectral entropy, nats)}, \tag{45}$$

$$D_{\text{eff}} = L \cdot \exp(H_{\text{mem}}) \quad \text{(effective memory dimension)}. \tag{46}$$

$D_{\text{eff}} = 1$ corresponds to a single-exponential (Markovian) kernel. $D_{\text{eff}} = L$ corresponds to a uniform spectrum—maximally non-Markovian. Power-law kernels drive $D_{\text{eff}} \to \infty$ as observation time grows, signalling infinite-dimensional memory.

10.2. D_{eff} as a Cognitive State Monitor

Computed at every timestep from the current kernel weights, $D_{\text{eff},n}(t)$ provides a real-time indicator of cognitive complexity at level n:

- $D_{\text{eff}} \approx 1$: System has collapsed to Markovian processing—insufficient memory for the task.
- $D_{\text{eff}} \approx L$: All channels contribute equally—high complexity, potentially approaching saturation (trigger growth).
- D_{eff} rising sharply: Sudden increase in task complexity, or instability onset.
- D_{eff} below threshold: Channels have specialised—trigger pruning of near-zero channels.

This interpretability is a *design property*, not an add-on: because the memory kernel is an explicit computational object with learnable SOE parameters, its spectral properties are computable at zero overhead beyond the weight normalisation in (45)–(46).

10.3. Comparison with Existing Architectures

Table 2 summarises HierAberConic-D2C against leading alternatives on properties that are either absent or non-interpretable in existing systems.

10.4. Renormalisation Consistency Validation

At each level transition, the following checks are performed:

1. $D_{\text{eff}}^{n+1} \le D_{\text{eff}}^{n} \cdot D_{\text{eff}}(K_{\text{bath}}^{n})$ (sub-multiplicative bound);
2. $M_{\text{scale}}^{n+1} \ge M_{\text{scale}}^{n}$ (span grows or stays constant);
3. $\bar{\gamma}^{n+1} < \bar{\gamma}^{n}$ (dominant rate slows at higher levels).

Violations signal either a modelling failure (missing slow channels) or a fitting artefact (spurious fast modes at a high level).

11. Stability: Full Proofs

11.1. Formulation A: Unconditional Stability

Theorem 11.1. Input-Driven Stability

For Formulation A (12) with $x(t) = 0$, the equilibrium is asymptotically stable for any $\ell, w_\ell, \gamma_\ell > 0$.

Proof. The Jacobian J_A is upper triangular with diagonal entries $\{-\ell, -\gamma_1, \ldots, -\gamma_L\}$. All eigenvalues are negative by assumption; stability follows from the eigenvalue condition. □ □

11.2. Formulation B: Lyapunov Analysis

Theorem 11.2. Negative-Feedback Stability (Lyapuno

For Formulation B (13) with $x(t) = 0$, define the Lyapunov function

$$V(u, \boldsymbol{\chi}) = \tfrac{1}{2}u^2 + \tfrac{1}{2}\sum_\ell \frac{w_\ell}{\gamma_\ell}\chi_\ell^2. \tag{47}$$

When condition (15) holds, $\dot{V} \le -c\,V$ for some $c > 0$, implying exponential stability.

Proof. Computing $\dot{V}$ along trajectories of

Table 2. Architectural comparison across key properties. ✓ = supported by design; ∼ = partially supported or heuristic; × = not supported.

Property	Trans-former	LSTM	S4/Mamba	Neural ODE	Pred. Coding	**HierAber-Conic-D2C**
Explicit memory kernel	×	×	∼	×	×	✓
Physical timescales	×	×	×	×	×	✓
Stability guarantee	×	×	×	×	×	✓
Hierarchical timescale org.	×	×	×	×	∼	✓
Predictive coding learning	×	×	×	×	✓	✓
Multi-timescale value learning	×	×	×	×	×	✓
Online continual learning	×	∼	×	×	∼	✓
No catastrophic forgetting	×	×	×	×	∼	✓
Interpretable diagnostics (D_{eff})	×	×	×	×	×	✓
Hardware-mappable (analog)	×	×	×	×	×	✓

(13):

$$\begin{aligned}\dot{V} &= u\dot{u} + \sum_\ell \frac{w_\ell}{\gamma_\ell}\chi_\ell\dot{\chi}_\ell \\ &= u\Big(-\ell u - \sum_\ell w_\ell\chi_\ell\Big) + \sum_\ell \frac{w_\ell}{\gamma_\ell}\chi_\ell(-\gamma_\ell\chi_\ell + u) \\ &= -\ell u^2 - \sum_\ell w_\ell\chi_\ell^2 + \sum_\ell w_\ell\chi_\ell u\Big(\frac{1}{\gamma_\ell} - 1\Big).\end{aligned}$$

For $\gamma_\ell \geq 1$, the cross term is non-positive and $\dot{V} \leq -\ell u^2 - \sum_\ell w_\ell\chi_\ell^2 \leq -2c\,V$ with $c = \min(\ell, \min_\ell \gamma_\ell)$. For general γ_ℓ, Young's inequality applied to the cross term yields $|\sum_\ell w_\ell\chi_\ell u| \leq \varepsilon u^2/2 + \sum_\ell \frac{w_\ell^2}{2\varepsilon}\chi_\ell^2$, and choosing ε to satisfy (15) gives $\dot{V} < 0$. □ □

11.3. Formulation C: Stability Condition

Proposition 11.1. Second-Order Stability

For Formulation C (14) with $\zeta > 0$, $\omega_0 > 0$, and $x(t) = 0$, the system is stable if

$$\sum_\ell \frac{w_\ell}{\gamma_\ell} < 2\zeta\omega_0. \tag{48}$$

11.4. CM Preservation Under Learning

Proposition 11.2. CM Invariance of Update Rules

The three-factor update rule (33), with the constraint $w_{n,\ell} \geq 0$ enforced after each step, and the timescale adaptation (34) with $\gamma_{n,\ell} \geq \gamma_{\min} > 0$, jointly preserve the CM structure of $K_n(t) = \sum_\ell w_{n,\ell}\, e^{-\gamma_{n,\ell}t}$.

Proof. A function is CM iff it is a positive linear combination of decaying exponentials with positive rates. The non-negativity constraint on $w_{n,\ell}$ and the positivity constraint on $\gamma_{n,\ell}$ maintain exactly this representation at every step. □ □

12. Experiments

The experiments reported here are initial validations establishing that the mathematical framework is implementable, numerically stable, and produces physically interpretable results. They should be read as proofs of concept confirming the theoretical foundations, not as comprehensive performance benchmarks. Broader benchmarks against state-of-the-art sequence models are underway.

12.1. Experiment 1: Coloured-Noise Approximation

12.1.1. Setup. We compare three stochastic processes over a $T = 500\,\text{s}$ horizon:

1. **White OU:** $\mathrm{d}x = -\theta x\,\mathrm{d}t + \sigma\,\mathrm{d}W$, $\theta = 1.0$,

$\sigma = 0.5$.

2. **Coloured OU (reference):** $\mathrm{d}x = -\theta x\,\mathrm{d}t + \eta\,\mathrm{d}t$; $\mathrm{d}\eta = -\alpha\eta\,\mathrm{d}t + \sigma\,\mathrm{d}W$, $\alpha = 0.1$.

3. **Aberconics (L=3):** $\mathrm{d}x = -\theta x\,\mathrm{d}t + \sum_i w_i\chi_i\,\mathrm{d}t$; $\mathrm{d}\chi_i = -\gamma_i\chi_i\,\mathrm{d}t + \sigma_i\,\mathrm{d}W_i$, log-spaced $\gamma \in \{0.01, 0.1, 1.0\}$.

The coloured OU autocorrelation function (ACF) is fitted using NNLS with 15 exponential basis functions ($\gamma \in [10^{-2}, 10^{1}]$, log-spaced).

12.1.2. Results. The NNLS fit recovers 7 significant SOE modes with normalised L^1 error $<$ 0.5% over 200 time lags—confirming that the two-timescale coloured OU ACF is well-represented by the SOE basis despite having only two underlying timescales. Memory persistence at lag $\tau = 50$ s: white OU $\rho(50) = 0.60$; coloured OU $\rho(50) = 0.99$; Aberconics $\rho(50) = 0.995$, demonstrating a 50$\times$ effective memory extension over baseline. Spectral diagnostics for the fitted kernel: $D_{\text{eff}} = 6.86$, $H_{\text{mem}} = 1.93$ nats, $M_{\text{scale}} = 2.29$ decades (Table 3).

Table 3. OU noise experiment: key metrics.

Metric	White OU	Coloured OU	Aberconics
$\rho(50)$	0.60	0.990	0.995
τ_{eff} (steps)	91	2052	4603
Memory extension	1$\times$	22$\times$	50$\times$
L^1 fit error	—	—	$<$0.5%
D_{eff}	1.0	—	6.86
M_{scale} (decades)	0.0	—	2.29
H_{mem} (nats)	0.0	—	1.93

12.1.3. Interpretation. The result validates three claims simultaneously: the NNLS kernel fitting procedure is accurate ($<$ 0.5% error); the SOE basis is expressive enough to represent multi-timescale coloured processes; and the spectral units correctly quantify the resulting memory structure. The $D_{\text{eff}} = 6.86$ out of $L = 7$ fitted modes indicates a nearly uniform spectral weight distribution—rich, distributed memory with no single dominant timescale.

12.2. Experiment 2: Lorenz '63 Chaos Suppression

12.2.1. Setup. The Lorenz system [16] with standard parameters ($\sigma = 10$, $\beta = 8/3$, $\rho_0 = 28$, $\lambda_{\max}^{\text{baseline}} \approx 0.9$) is augmented with Formulation B memory feedback on the x-equation:

$$\dot{x} = \sigma(y - x) - \sum_{\ell=1}^{3} w_\ell \chi_\ell, \qquad \dot{\chi}_\ell = -\gamma_\ell \chi_\ell + x. \tag{49}$$

Parameters $(w_1, w_2, w_3, \gamma_1, \gamma_2, \gamma_3)$ are optimised via BFGS to minimise the largest Lyapunov exponent $\lambda_{\max}$, estimated by the variational method with threaded multi-IC evaluation (4-core, 5.7$\times$ speedup). Training uses 2 initial conditions (ICs) simultaneously; 3 held-out ICs are used for validation.

12.2.2. Results. Optimised parameters (Table 4) show decay rates $\gamma \approx (0.065, 0.018, 0.008)$, corresponding to timescales $\tau \approx (15, 56, 119)$ s—spanning two orders of magnitude, matching the multi-timescale structure of Lorenz dynamics. Lyapunov exponent is reduced by 25% on training ICs. On validation: 2 of 3 ICs achieve periodic stability ($\lambda < 0$); 1 IC (high initial $z = 20$) remains chaotic. The failure case reflects geometric basin-of-attraction constraints, not a framework limitation, and is consistent with theoretical predictions [20].

Table 4. Lorenz chaos suppression: parameter and Lyapunov comparison.

Metric	Baseline	Optimised
λ_{train}	-0.019	-0.024 (-25%)
γ_1	0.200	0.065
γ_2	0.050	0.018
γ_3	0.010	0.008
w_1	0.500	0.035
w_2	0.200	0.019
w_3	0.100	0.051
Training ICs stable	—	2/2
Validation ICs stable	—	2/3
Wall-clock time	—	130 s

12.2.3. Interpretation. Three claims are validated: (1) the memory-augmented dynamical system is numerically stable with no blow-ups across 200 optimisation evaluations; (2) multi-timescale memory feedback can suppress chaotic dynamics, demonstrating the "memory as intelligent dissipation" paradigm; (3) optimised parameters are physically interpretable— γ values that span two decades correspond naturally to the separation of fast oscillation and slow drift in Lorenz dynamics.

The 2/3 validation success rate motivates the IC-aware training strategy (cluster-specific parameter optimisation) described as a direct next step.

12.3. Planned Experiments

The following experiments are in preparation and will be reported in subsequent work:

Multi-timescale prediction. Synthetic signals composed of three superimposed sinusoids at timescales spanning two orders of magnitude. Expected: level specialisation visible in per-level D_{eff} and M_{cap}.

Delayed-reward control. Action at $t = 0$, reward only at $t = T_{\text{delay}}$. Expected: the memory channel with $\gamma_\ell \approx 1/T_{\text{delay}}$ dominates after training, providing interpretable credit assignment.

Continual learning without forgetting. Sequential task pairs with different characteristic timescales. Expected: timescale separation prevents interference between tasks in distinct channels.

Long-context sequence modelling. Synthetic sequences requiring integration over 500+ steps. Comparison against S4, Mamba, and LSTM on accuracy and D_{eff} diagnostic interpretability.

Gray–Scott reaction-diffusion. Memory feedback modulating pattern formation parameters. Expected: D_{eff} maps showing spatially distributed memory complexity aligned with pattern boundaries.

A. Pseudocode Reference

A.1. Single AMB Step

Algorithm 4 AMBStep: single Aberconics Memory Block forward step

Input: State $(u, \boldsymbol{\chi})$, forcing x, parameters $(\ell, \boldsymbol{w}, \boldsymbol{\gamma})$, timestep Δt, formulation $\in \{A, B, C\}$
Output: Updated state $(u', \boldsymbol{\chi}')$, spectral units $\mathcal{S}$

Compute memory feedback: $M \leftarrow \sum_\ell w_\ell \chi_\ell$
if formulation $= A$ **then**
  $\dot{u} \leftarrow -\ell u + M + x$
  $\dot{\chi}_\ell \leftarrow -\gamma_\ell \chi_\ell + x$ $\forall \ell$
else if formulation $= B$ **then**
  $\dot{u} \leftarrow -\ell u - M + x$
  $\dot{\chi}_\ell \leftarrow -\gamma_\ell \chi_\ell + u$ $\forall \ell$
else if formulation $= C$ **then**
  $\ddot{u} \leftarrow -2\zeta\omega_0 \dot{u} - \omega_0^2 u + M + x$
  $\dot{\chi}_\ell \leftarrow -\gamma_\ell \chi_\ell + u$ $\forall \ell$
end if
Euler integrate: $u' \leftarrow u + \dot{u}\,\Delta t$; $\chi'_\ell \leftarrow \chi_\ell + \dot{\chi}_\ell\,\Delta t$
Check stability: assert $\ell > \sum_\ell w_\ell / \gamma_\ell$
Compute $\mathcal{S} = \text{SpectralUnits}(\boldsymbol{w}, \boldsymbol{\gamma})$
return $(u', \boldsymbol{\chi}'), \mathcal{S}$

A.2. Hierarchy Step

Algorithm 5 HierarchyStep: one timestep of the full hierarchical system

Input: Observation x_{raw}, all level states $\{\boldsymbol{z}_n\}$, Δt
Output: Updated states $\{\boldsymbol{z}'_n\}$, prediction errors $\{\varepsilon_n\}$

Bottom-up pass:
for $n = 1$ to N **do**
  $x_n \leftarrow x_{\text{raw}}$ if $n = 1$ else $h_n(u_{n-1})$
  Compute top-down modulation: $(\boldsymbol{w}_n, \boldsymbol{\gamma}_n) \leftarrow \left(e^{\alpha(u_{n+1})}, e^{\beta(u_{n+1})}\right)$ (if $n < N$)
  $(u'_n, \boldsymbol{\chi}'_n) \leftarrow \text{AMBStep}(u_n, \boldsymbol{\chi}_n, x_n, \ell_n, \boldsymbol{w}_n, \boldsymbol{\gamma}_n, \Delta t, B)$
end for
Prediction errors:
for $n = 2$ to N **do**
  $\hat{y}_n \leftarrow W_{\text{pred},n}\, u'_n$
  $\varepsilon_n \leftarrow u'_{n-1} - \hat{y}_n$
end for
return $\{(u'_n, \boldsymbol{\chi}'_n)\}, \{\varepsilon_n\}$

A.3. Three-Factor Weight Update

Algorithm 6 UpdateWeights: three-factor Hebbian–TD update

Input: Level state χ_n, prediction error ε_n, TD errors $\{\delta_{n,\ell}\}$, current weights w_n, rates $\eta_{\text{pred}}, \eta_{\text{val}}, \beta_0, \Delta t$
Output: Updated weights w'_n

1: **for** $\ell = 1$ to L_n **do**
2: $\quad \beta \leftarrow \beta_0 / (1 + |\varepsilon_n|)$
3: $\quad \Delta w_\ell \leftarrow \eta_{\text{pred}}\, \chi_{n,\ell}\, \varepsilon_n + \eta_{\text{val}}\, \chi_{n,\ell}\, \delta_{n,\ell} - \beta\, w_{n,\ell}$
4: $\quad w'_{n,\ell} \leftarrow \max\big(0,\ w_{n,\ell} + \Delta w_\ell\, \Delta t\big)$ *// enforce CM*
5: **end for**
6: **return** w'_n

A.4. Kernel Fitting via NNLS

Algorithm 7 FitSOEKernel: NNLS-based SOE kernel identification

Input: Target ACF $\hat{K}(\tau)$ on lags $\{\tau_i\}$, basis rates $\{\gamma_j\}$ (log-spaced, $j = 1 \dots M$), pruning threshold ϵ
Output: SOE weights w, rates γ, fit error

1: Build design matrix: $A_{ij} \leftarrow e^{-\gamma_j \tau_i}$
2: Solve NNLS: $w \leftarrow \arg\min_{w \geq 0} \left\| A w - \hat{K} \right\|^2$
3: Prune: keep j where $w_j > \epsilon \cdot \max_j w_j$
4: Normalise: $w \leftarrow w / \sum_j w_j$
5: Compute spectral units: $\mathcal{S} \leftarrow \text{SpectralUnits}(w, \gamma)$
6: Compute fit: $L^1 \leftarrow \left\| A w - \hat{K} \right\|_1 / \left\| \hat{K} \right\|_1$
7: **return** γ, w, L^1, $\mathcal{S}$

B. Hardware Mapping

Each memory channel χ_ℓ with γ_ℓ maps to an RC circuit: $R_\ell = 100\,\text{k}\Omega$, $C_\ell = 1/(\gamma_\ell \cdot R_\ell)$. A summing operational amplifier with feedback resistors $R_{f,\ell} = R_\ell / w_\ell$ realises the weighted sum. A 7-channel implementation fits on a $< 5\,\text{cm}^2$ PCB; memristor-based implementations [19] could reduce this to $< 1\,\text{mm}^2$.

Neuromorphic mapping (Intel Loihi 2): each χ_ℓ maps to a dendritic compartment with time constant $\tau_\ell = 1/\gamma_\ell$; w_ℓ maps to synaptic weight; ℓ_n maps to the membrane leak. The correspondence between the continuous SOE equations and the discrete-time Loihi 2 compartment model is exact to first-order Euler discretisation.

C. Hyperparameter Guide

Table 5. Hyperparameter guide and typical ranges.

Parameter	Symbol	Typical range	Notes
Channels per level	L_n	3–12	Start with 5; grow via D_{eff}
Decay rates	γ_ℓ	$[10^{-2}, 10^{1}]$	Log-spaced; adapt via (34)
Leak rate	ℓ_n	0.05–0.5	Must satisfy (15)
Pred. learning rate	η_{pred}	10^{-3}–10^{-2}	Per level
Value learning rate	η_{val}	10^{-4}–10^{-3}	Lower than pred.
Timescale adapt.	α	10^{-5}–10^{-4}	$\alpha \ll \eta_{\text{pred}}$
Decay baseline	β_0	10^{-4}–10^{-3}	Error-modulated
Consolidation rate	β_c	10^{-3}–10^{-2}	Periodic
Diversity penalty	λ_{div}	0.01–0.1	Prevents γ collapse
Prune threshold	ϵ_{prune}	0.01	Fraction of max weight
Window size	W	50–500	Task-dependent

D. Implementation Architecture

D.1. Repository Structure

The reference implementation is available at `github.com/pilloverx/aberconics-framework`. The codebase follows a three-layer design:

Layer 1 — GFE Core (C++). Kernel factory: NNLS solver, SOE basis, spectral units, parameter packing. Language-agnostic via C ABI; accessed from Python via `ctypes` and from Julia via `ccall`.

Layer 2 — ABERSOE Runtime (C++ / Julia). Stateful dynamical system: memory channels, coupling formulations, Hebbian updates, Lorenz/OU/Gray-Scott/echo scenarios. Julia reference experiments are production-ready (`code/julia/examples/`).

Layer 3 — Hierarchical MIN (C++ / Python). Multi-level coupling (`hierarchical_min.cpp`), top-down modulation (`hierarchical_min_coupling.cpp`), renormalisation (`hierarchical_min_renorm.cpp`), cross-level diagnostics (`hierarchical_min_diagnostics.cpp`).

Training Runtime (Python, in development). AberconicsDirector, TraceStore, StabilityMonitor, SpectralInspector, CheckpointManager. Orchestrates the C++ substrate; interfaces with PyTorch for value head parameter updates.

References

[1] H. Mori, "Transport, collective motion, and Brownian motion," *Prog. Theor. Phys.*, vol. 33, pp. 423–455, 1965.

[2] R. Zwanzig, "Memory effects in irreversible thermodynamics," *Phys. Rev.*, vol. 124, pp. 983–992, 1961.

[3] W. McLean and V. Thomée, "Numerical solution of an evolution equation with a positive-type memory term," *J. Aust. Math. Soc. Ser. B*, vol. 35, pp. 23–70, 1993.

[4] Y. Nakatsukasa, O. Sète, and L. N. Trefethen, "The AAA algorithm for rational approximation," *SIAM J. Sci. Comput.*, vol. 40, pp. A1494–A1522, 2018.

[5] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *ICLR*, 2022.

[8] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv:2312.00752*, 2023.

[9] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *NeurIPS*, 2018.

[10] E. Dupont, A. Doucet, and Y. W. Teh, "Augmented neural ODEs," in *NeurIPS*, 2019.

[11] R. P. N. Rao and D. H. Ballard, "Predictive coding in the visual cortex," *Nature Neurosci.*, vol. 2, pp. 79–87, 1999.

[12] K. Friston, "The free-energy principle: a unified brain theory?" *Nature Rev. Neurosci.*, vol. 11, pp. 127–138, 2010.

[13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[14] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, pp. 181–211, 1999.

[15] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in *ICLR*, 2017.

[16] E. N. Lorenz, "Deterministic nonperiodic flow," *J. Atmos. Sci.*, vol. 20, pp. 130–141, 1963.

[17] S. Diekelmann and J. Born, "The memory function of sleep," *Nature Rev. Neurosci.*, vol. 11, pp. 114–126, 2010.

[18] D. Attwell and S. B. Laughlin, "An energy budget for signaling in the grey matter of the brain," *J. Cereb. Blood Flow Metab.*, vol. 21, pp. 1133–1145, 2001.

[19] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, pp. 80–83, 2008.

[20] D. K. Ahorlu, "Stabilizing memory kernels: Corrected formulations for sum-of-exponentials non-Markovian dynamics with experimental validation," *Preprint*, 2026.

[21] D. K. Ahorlu, "Geometric Fourier extension: A unified framework for spectral-temporal methods," *Preprint*, 2025.